

FOL-Zinc Large-Integer Performance: Integer Width vs Random-Field Width

Technical note for cryptographic review

2026-06-25

Executive summary

The current FOL-Zinc adapter appears to pay a prohibitive performance cost for large exact integers because it couples two parameters that should conceptually be distinct:

1. **Exact integer representation width:** how many machine limbs are needed to hold explicit integer witness/constant values such as `F_10000`.
2. **Random-field / soundness profile:** how large the sampled prime field is for the probabilistic Zinc proof.

For examples such as exact, non-modular Fibonacci `F_10000`, the current runner has been using a very wide profile such as:

`Int<128> / RandomField<256>`

The `Int<128>` part is understandable as an implementation representation choice: `F_10000` is about 6,942 bits, and a 128-limb, 64-bit-per-limb container can hold roughly 8,192 bits. The problematic part is automatically pairing this with `RandomField<256>`, which appears to force a roughly 16,384-bit random field profile. That makes prime sampling, finite-field arithmetic, and dense proof objects extremely expensive.

Mathematically, this coupling is not the point of Zinc. Zinc is meant to prove integer/rational relations by reducing them modulo sampled primes, with soundness governed by the probability that a false integer relation vanishes modulo the sampled primes. It is not meant to require a field modulus larger than every integer value appearing in the computation.

The likely high-impact optimisation is therefore:

Use a wide integer representation only where exact host-side materialisation requires it, but use a much smaller sampled random field profile for the Zinc proof.

Concretely, for `F_10000`, the desired experiment is something like:

```
Int<128> / RandomField<4>
```

rather than:

```
Int<128> / RandomField<256>
```

This is expected to reduce the largest field-side dense allocation estimate by a factor on the order of 64 in the current estimator. For the compact `F_10000` benchmark, the rough estimate would change from hundreds of GiB to something closer to the tens-of-GiB range. That is still large, but it is a different class of problem.

This note separates the mathematical question from the current Rust implementation question and lists concrete next engineering steps.

Context

The FOL-Zinc paper describes a route:

```
FOL witness semantics
-> enriched polynomial form
-> mkQ_x^F(<phi>)
-> beta_F evaluation
-> integer polynomial / AIR relation
-> Zinc-compatible proof
```

The paper states that Zinc is intended to reduce arithmetisation overhead for relations over integers or mixed-characteristic settings by sampling random primes and running finite-field checks modulo those primes, with low soundness error. In the paper’s description, this removes the need to rewrite the user-level integer relation into a single fixed finite field relation, beyond reduction modulo primes. See the introductory discussion of Zinc in `fol-zinc.pdf`, especially the paragraph beginning “Zinc allows for succinct proofs...”.

The paper also states that Zinc relations are defined with a bit-length bound, conventionally written `delta`, on coefficients/inputs. In Corollary 4.4, the final FOL-to-Zinc construction inherits the Zinc soundness error and includes `delta` as part of the global parameters. Thus `delta` is a relation-size / magnitude-bound parameter, not simply a demand that the sampled field be larger than every integer value.

The performance problem observed

The problematic case is exact, non-modular Fibonacci, for example:

```
./zkfol_zinc_adapter/folzinc bench fib --n 10000 --run --repeat 1 --int-limbs 128
```

The benchmark is intentionally not RISC Zero's modulo- 2^{64} Fibonacci. It is meant to prove the exact integer value of `F_10000`.

Relevant facts:

<code>F_10000</code> bit length:	about 6,942 bits
compact constraints:	about 10,001
compact witness vars:	about 10,000
padded Zinc dimension:	about 16,384

The current package historically treated `--int-limbs 128` as selecting the entire paired profile:

```
Int<128> / RandomField<256>
```

This is much more expensive than merely storing 6,942-bit integers. It also makes Zinc sample and work over a very large random field. On the user's machine, this effectively did not terminate for small tests when an oversized field profile was forced, and it is expected to be prohibitive for `F_10000`.

The conceptual distinction

The key distinction is:

Large exact integer values do not necessarily imply an equally large sampled prime field.

If an integer relation is false over the integers, then after reducing modulo a sampled prime it may accidentally become true only if the nonzero integer obstruction vanishes modulo that prime. Zinc's soundness analysis is about making this probability small by choosing parameters appropriately. This is not the same as selecting a prime larger than every witness value to avoid all wraparound.

In other words, the correct conceptual pipeline is:

```
large exact integer relation
-> reduce relation modulo sampled prime(s)
-> prove reduced finite-field relation
-> soundness error bounds probability of bad modular coincidence
```

not:

```
large exact integer relation
-> choose field larger than every integer
-> avoid all modular reduction effects
```

This matters because avoiding all modular reduction effects for values like `F_10000` forces enormous finite fields, whereas Zinc's intended advantage is to avoid that requirement.

Current implementation issue

The public Rust Zinc implementation uses fixed type families. In the source, the `implement_random_field_zip_types!` macro maps a chosen integer profile parameter `N` to associated types of the form:

```
type N = Int<N>
type L = Int<2N>
type K = Int<4N>
type M = Int<8N>
```

The current FOL-Zinc runner dispatches concrete paired profiles such as:

```
Int<2>    / RandomField<4>
Int<4>    / RandomField<8>
Int<8>    / RandomField<16>
Int<16>   / RandomField<32>
Int<32>   / RandomField<64>
Int<64>   / RandomField<128>
Int<128>  / RandomField<256>
```

That pairing was introduced to satisfy Rust trait bounds and make the public proof-of-concept build reliably. It is an implementation dispatch strategy, not a cryptographic necessity.

The current Zinc prime generator also draws a number of bytes proportional to the selected field word count. Thus a larger `RandomField<F>` directly increases prime sampling size and field arithmetic cost. This explains why forcing `RandomField<256>` is expensive even for a tiny relation such as `F_3 = 2`.

The current Zinc code also includes a `FieldMap` path that maps larger integer objects into a random field by reduction modulo the field modulus. That is encouraging: it suggests that decoupling the integer representation width from the sampled-field width may be possible, at least conceptually and perhaps with a small runner-side or Zinc-side patch.

However, the current Rust trait bounds may not permit every cross-pairing directly. This needs to be tested. If direct compilation fails, the runner may need an explicit adapter layer that reduces wide host-side integers into the smaller field type before calling Zinc's prover.

Why `--int-limbs 128` was toxic

The observed bad behaviour came from mixing two costs:

1. Storing exact 6,942-bit Fibonacci values.
2. Proving over a roughly 16,384-bit sampled field.

The first cost is acceptable or at least plausible for a research benchmark. Ten thousand 7k-bit integers are not free, but they are not inherently impossible.

The second cost is the killer. It inflates prime generation, finite-field arithmetic, and dense proof-side objects.

A rough memory estimate for compact F₁₀₀₀₀ is:

```
padded dimension      ~= 16,384
field-side dense cells ~= 16,384^2
```

If the per-cell cost scales with the selected field profile, then switching from `RandomField<256>` to `RandomField<4>` reduces that component by roughly a factor of 64.

Using the current estimator’s rough model:

```
RandomField<256>: hundreds of GiB, around 768 GiB for one large dense object
RandomField<4>:   tens of GiB, around 12 GiB for the corresponding object
```

These are approximate engineering estimates, not cryptographic parameters. But they explain the practical difference between “does not terminate” and “possibly benchmarkable on a large workstation”.

Concrete engineering proposal

The next package change should separate representation width and proof field width.

Proposed CLI model

At the high level:

```
./zkfol_zinc_adapter/folzinc bench fib --n 10000 --run --repeat 1 \
  --representation-limbs 128 \
  --field-limbs 4
```

or, better for normal users:

```
./zkfol_zinc_adapter/folzinc bench fib --n 10000 --run --repeat 1 \
  --soundness-bits 128
```

where the wrapper chooses representation and field sizes separately.

A practical design would be:

```
representation-limbs:
    smallest profile that can hold explicit integers materialised by the adapter
field-limbs:
    profile chosen for the sampled random field / soundness target
soundness-bits:
    user-facing target from which field-limbs and repetitions may be derived
```

Proposed internal model

Separate these quantities in the export metadata and runner preflight:

```
value_bits:
    largest explicit integer value in JSON / witness / constants
relation_bound_bits:
    bound or estimate for intermediate integer relation values
representation_bits:
    host-side fixed-width container capacity
field_bits:
    sampled-prime field size
soundness_target_bits:
    desired statistical / knowledge-soundness target
```

Currently these are blurred into `--int-limbs`.

Proposed proof experiment

Test whether the public Zinc proof-of-concept can compile and verify a cross-profile run:

```
integer CCS type:      Int<128>
random field type:     RandomField<4>
```

If yes, benchmark compact exact Fibonacci with:

```
folzinc bench fib --n 1000 --run --repeat 3 \
    --representation-limbs 128 --field-limbs 4
folzinc bench fib --n 10000 --run --repeat 1 \
    --representation-limbs 128 --field-limbs 4
```

If no, implement an explicit import/reduction adapter:

1. Parse large decimal/JSON integers using arbitrary precision or `Int<128>`.
2. Sample the smaller field modulus.
3. Reduce each integer value modulo that modulus.
4. Build field-side CCS / witness data directly in `RandomField<4>`.
5. Pass the reduced field relation to Zinc's prover.

The adapter must preserve the Zinc transcript/statement semantics; this should be reviewed carefully.

Alternative: CRT / residue certificate mode

Another route is to avoid storing huge exact integers in the proof at all.

For exact Fibonacci, prove enough modular projections:

```
F_n mod p_1
F_n mod p_2
```

...
 $F_n \bmod p_k$

with the product bound:

$P = p_1 * p_2 * \dots * p_k > \text{known upper bound on } F_n$

Then the residues uniquely determine the exact integer F_n by the Chinese remainder theorem.

This remains an exact, non-modular proof of the integer value, because the collection of residues pins down the unique integer in the known range. But every in-proof value can remain small.

Advantages:

- avoids huge integer witness values;
- keeps representation and field sizes small;
- may be more robust against current Rust fixed-width limitations.

Disadvantages:

- more constraints, because many residue computations must be proven;
- not the same benchmark shape as a single exact-integer trace;
- needs careful public-output binding and CRT-bound documentation.

This is a good fallback if `Int<128> / RandomField<4>` is hard to make compile cleanly in the current Zinc proof-of-concept.

Alternative: fast-doubling Fibonacci

The current compact Fibonacci benchmark proves a length- n recurrence trace:

$F_1, F_2, \dots, F_n$

For $n = 10000$, that gives about 10,000 recurrence constraints.

If the goal is to prove F_{10000} as fast as possible, a fast-doubling recurrence is much better:

$$\begin{aligned} F(2k) &= F(k) * (2F(k+1) - F(k)) \\ F(2k+1) &= F(k+1)^2 + F(k)^2 \end{aligned}$$

This gives a trace length around $\log_2(n)$ rather than n . For $n = 10000$, this is about 14 decomposition steps.

This is not a direct apples-to-apples comparison with a zkVM running a simple Fibonacci loop, but it is the right benchmark for “best FOL-Zinc formulation of exact Fibonacci”. It should be considered separately from the direct zkVM comparison benchmark.

Cryptographic questions to ask

1. **Soundness parameterisation:** Given the Zinc soundness theorem, how should `field-limbs`, number of sampled primes, and repetition count map to a user-facing `soundness-bits` parameter?
2. **Relation-bound dependence:** Does the soundness bound depend on the bit length of the false integer obstruction, the degree/number of polynomials, or both? How should the runner estimate the relevant bound for generated CCS/R1CS instances?
3. **Cross-profile validity:** Is `Int<128>` / `RandomField<4>` sound and compatible with Zinc’s proof as implemented, assuming explicit modular reduction is done correctly?
4. **Transcript binding:** If the runner reduces wide integer data into a smaller sampled field before proving, what exactly must be committed or transcript-bound to ensure the verifier is checking the intended integer relation rather than only an underspecified modular artefact?
5. **Public constants:** For exact public constants such as `F_10000`, is it preferable to materialise the full decimal integer in the statement, to commit to a hash plus range/CRT certificate, or to expose enough modular residues to bind the value?
6. **CRT mode:** Would a CRT residue certificate be considered faithful to the “exact non-modular” benchmark claim, provided the product of moduli exceeds a public upper bound on `F_n`?
7. **Benchmark fairness:** For comparison with RISC Zero’s modulo- 2^{64} Fibonacci benchmark, should FOL-Zinc report both a direct exact-integer trace and a best-formulation fast-doubling/CRT proof?

Recommended next steps

1. **Documentation fix:** Stop describing `--int-limbs` as if it must exceed the largest mathematical value for cryptographic reasons. It is currently an implementation representation/profile knob.
2. **Runner experiment:** Try to compile and run a cross-profile runner with:

`Int<128>` / `RandomField<4>`

3. **Preflight improvement:** Report separate estimates for:

`representation width`
`sampled-field width`
`estimated dense field allocation`
`relation-bound bits`

4. **CLI improvement:** Add:

```
--representation-limbs
--field-limbs
--soundness-bits
```

and retain `--int-limbs` only as a backwards-compatible alias for representation width, with a warning.

5. **Fallback design:** Add a CRT/residue mode for exact Fibonacci if direct cross-profile mapping is blocked by current Zinc trait constraints.

6. **Benchmark plan:** Report three categories separately:

- A. direct exact integer recurrence trace;
- B. exact integer recurrence with decoupled field profile;
- C. optimised exact proof, e.g. fast doubling or CRT residues.

Provisional conclusion

The current `--int-limbs 128` nontermination is likely avoidable. It appears to be caused mainly by using an unnecessarily huge sampled random field profile, not by the mere presence of large exact integers.

The most promising immediate optimisation is to decouple host integer representation from random-field size:

```
large exact integer container where needed,
small/moderate sampled prime field for proof,
soundness controlled by Zinc's modular reduction analysis.
```

For `F_10000`, this could plausibly reduce the worst field-side memory estimate from hundreds of GiB to the tens-of-GiB range before deeper algorithmic improvements. This still needs implementation and cryptographic review, especially around transcript binding and soundness parameterisation.

References

1. Gabbay and Mendelsohn, `fol-zinc.pdf`, especially the Zinc introduction, Definition 2.9, Lemma 4.2, Corollary 4.4, Remarks 4.5-4.6, and Appendix A.
2. NethermindEth Zinc repository README, current public proof-of-concept implementation: <https://github.com/NethermindEth/zinc>
3. Zinc source, `src/field/int.rs`, especially `implement_random_field_zip_types!`: <https://raw.githubusercontent.com/NethermindEth/zinc/main/src/field/int.rs>
4. Zinc source, `src/field.rs`, especially `RandomField` and `FieldMap` mapping: <https://raw.githubusercontent.com/NethermindEth/zinc/main/src/field.rs>

5. Zinc source, `src/prime_gen.rs`, especially prime sampling width from `F::W::num_words() * 8`: https://raw.githubusercontent.com/NethermindEth/zinc/main/src/prime_gen.rs